

Spring Transaction

Spring @Transactional configuration.

Here's what **each attribute means**, one by one:

propagation = Propagation.NEVER

Transaction propagation behavior.

- **NEVER** means:
 - The method **must NOT run inside a transaction**.
 - If a transaction **already exists**, Spring will **throw an exception** (IllegalTransactionStateException).

Use when you want to ensure the method always runs **non-transactionally**.

Use Case	Propagation	Isolation	Explanation
Normal service	REQUIRED	READ_COMMITTED	Joins existing transaction or creates one; prevents dirty reads
Logging / Audit	REQUIRES_NEW	READ_COMMITTED	Always commits logs independently of business transaction
Read-only operations	SUPPORTS	READ_COMMITTED	Uses transaction if present; otherwise runs non-transactionally
External API calls	NEVER	N/A	Ensures no DB transaction is active

Financial operations	REQUIRED	SERIALIZABLE	Strongest consistency, prevents all read anomalies
-----------------------------	----------	--------------	--

isolation = Isolation.READ_COMMITTED

Transaction isolation level (how visible data changes are between transactions).

- **READ_COMMITTED** means:
 - A transaction can only read **committed data**.
 - Prevents **dirty reads** (reading uncommitted changes from another transaction).
 - Does **not** prevent:
 - Non-repeatable reads
 - Phantom reads

This is the **default isolation level** in many databases (e.g., PostgreSQL, Oracle).

What is a Dirty Read?

Reading data that was **modified by another transaction but not yet committed**.

Isolation vs Read Problems

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read
READ_UNCOMMITTED	YES	YES	YES
READ_COMMITTED	NO	YES	YES
REPEATABLE_READ	NO	NO	(DB-dependent)
SERIALIZABLE	NO	NO	NO

readOnly = false

Indicates whether the transaction is read-only.

- false means:
 - The transaction **can modify data** (INSERT, UPDATE, DELETE).
- If set to true:
 - Spring may optimize for read performance.
 - Some databases may reject write operations.

Use true for query-only methods to improve performance and safety.

timeout = 10

Maximum time (in seconds) the transaction is allowed to run.

- If the transaction takes **more than 10 seconds**:
 - Spring will **roll back** the transaction.
 - A TransactionTimeoutException is thrown.

Helps prevent long-running or stuck transactions.

rollbackFor = {CustomException.class}

Defines which exceptions trigger a rollback.

- By default, Spring rolls back on:
 - RuntimeException
 - Error
- This setting adds:
 - Rollback when CustomException (even if it's a **checked exception**) is thrown.

Useful when you want checked exceptions to cause rollback.

Summary Table

Attribute	Meaning
propagation = NEVER	Method must run without a transaction
isolation = READ_COMMITTED	Can only read committed data

readOnly = false	Allows data modification
timeout = 10	Transaction must finish within 10 seconds
rollbackFor = CustomException.class	Roll back when CustomException is thrown
